

Machine Learning and Pattern Recognition

Lecture 1 - Introduction and Nearest Neighbors

Mamdouh Farouk

Agenda

1. Introduction
2. What is Machine Learning?
3. Examples of Machine Learning
4. Why This Class?
5. Nearest Neighbor Methods

Marking Scheme

Component	% Final Grade
Assignments	15%
Practical exam	15%
Project	10%
Midterm Exam	20%
Final Exam	40%

There are lots of freely available, high-quality ML resources.

Here are some recommended textbooks.

- Bishop: Pattern Recognition and Machine Learning.
- Amin Zollanvari, Machine Learning with Paython; theory and implementation.

What is Learning?

"The activity or process of gaining knowledge or skill by studying, practicing, being taught, or experiencing something."

Merriam Webster dictionary

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ."

Tom Mitchell

What is Machine Learning?

- For many problems, it's difficult to program the correct behavior by hand
 - ▶ recognizing people and objects
 - ▶ understanding human speech
- Machine learning approach: program an algorithm to automatically learn from data, or from experience
- Why might you want to use a learning algorithm?
 - ▶ hard to code up a solution by hand (e.g. vision, speech)
 - ▶ system needs to adapt to a changing environment (e.g. spam detection)
 - ▶ want the system to perform *better* than the human programmers
 - ▶ privacy/fairness (e.g. ranking search results)

Types of Machine Learning

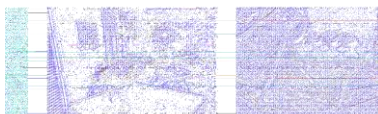
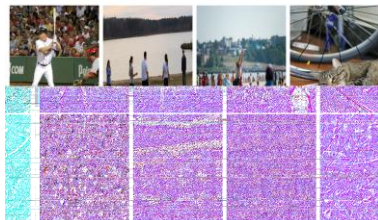
- ❖ **Supervised learning:** have labeled examples of the correct behavior
- ❖ **Unsupervised learning:** no labeled examples – instead, looking for “interesting” patterns in the data
- ❖ **Reinforcement learning:** learning system (agent) interacts with the world and learns to maximize a scalar reward signal

Computer Vision

Object detection, semantic segmentation, pose estimation, and almost every other task is done with ML.



Figure 4. More results of Mask R-CNN on COCO test images, using ResNet-101-499x and running at 5 fps, with 35.7 mask AP (Table 1).



DAQAR 1583
What is there in front of the
sofa?
Ground truth: table
IMG-BOW:
2-VIS-BLSTM:
LSTM: chair (0.47)

COCOQA 5078
How many leftover donut
the red bicycle holding?
Ground truth: three
IMG-BOW: two (0.57)
2-VIS-BLSTM:
BOW: one (0.29)

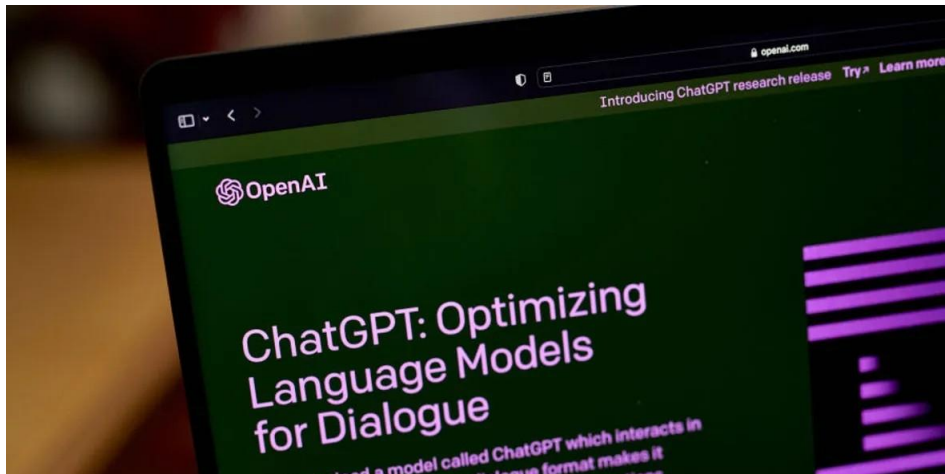


Instance segmentation -

[Link](#)

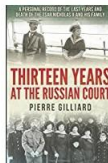
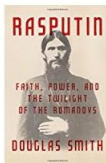
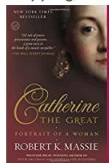
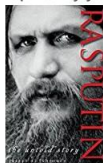
Natural Language Processing

Machine translation, sentiment analysis, topic modeling, spam filtering, general purpose chatbots (ChatGPT/GPT4).

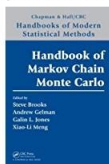
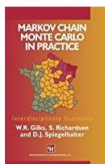


E-commerce & Recommender Systems : Amazon, Netflix, ...

Inspired by your shopping trends



Related to items you've viewed [See more](#)



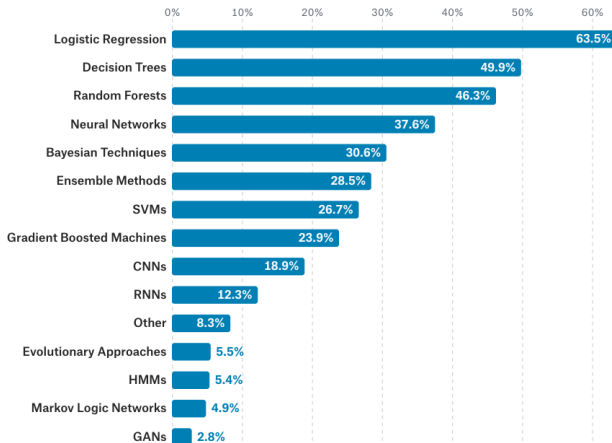
2 Why study Machine Learning

Why This Class?

- Automating data analysis and prediction
- Enabling you to build models that forecast future outcomes based on historical data
- The principles you learn in this course will be essential to understand and apply neural nets.
- The techniques in this course are the first things to try for a new ML problem.
 - ▶ E.g., try logistic regression before building a deep neural net!

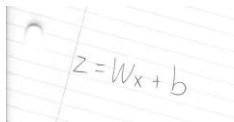
Why This Class?

2017 Kaggle survey of data science and ML practitioners:
What data science methods do you use at work?



Implementing Machine Learning Systems

- Derive an algorithm (with pencil and paper), and then translate the math into code.
- Array processing (NumPy)
 - ▶ **vectorize** computations (express them in terms of matrix/vector operations) to exploit hardware efficiency
 - ▶ This also makes your code cleaner and more readable!



A photograph of a piece of lined paper with the equation $z = Wx + b$ handwritten in black ink.

```
z = np.zeros(m)
for i in range(m):
    for j in range(n):
        z[i] += W[i, j] * x[j]
    z[i] += b[i]
```

$$z = W @ x + b$$

Implementing Machine Learning Systems

- Neural net frameworks: PyTorch, TensorFlow, JAX, etc.
 - ▶ automatic differentiation
 - ▶ compiling computation graphs
 - ▶ libraries of algorithms and network primitives
 - ▶ support for graphics processing units (GPUs)
- Why take this class if these frameworks do so much for you?
 - ▶ So you know what to do if something goes wrong!
 - ▶ Debugging learning algorithms requires sophisticated detective work, which requires understanding what goes on beneath the hood.
 - ▶ That's why we derive things by hand in this class!

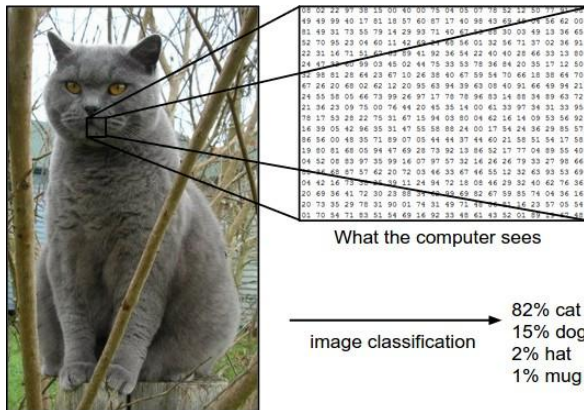
Introduction

- Today (and for much of this course) we focus on **supervised learning**.
- This means we are given a **training set** consisting of **inputs** and corresponding **labels**, e.g.

Task	Inputs	Labels
object recognition	image	object category
image captioning	image	caption
document classification	text	document category
speech-to-text	audio waveform	text
:	:	:

Input Vectors

What an image looks like to the computer:



[Image credit: Andrej Karpathy]

Input Vectors

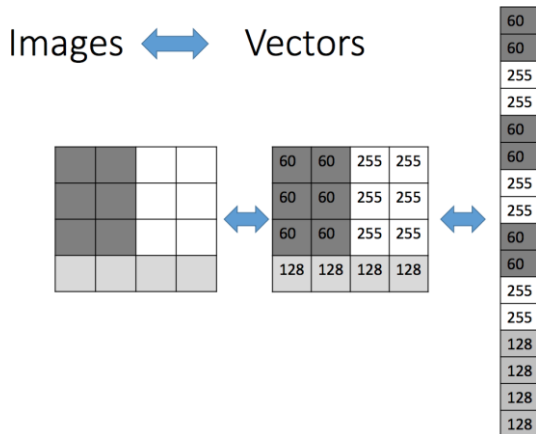
Machine learning algorithms need to handle lots of types of data: images, text, audio waveforms, credit card transactions, etc.

Common strategy: represent the input as an **input vector** in $\mathbb{R}^d$

- ▶ **Representation** = mapping to another space that's easy to manipulate
- ▶ Vectors are a great representation since we can do linear algebra!

Input Vectors

Can use raw pixels:



Can do much better if you compute a vector of meaningful features.

- Mathematically, our training set consists of a collection of pairs of an input vector $\mathbf{x} \in \mathbb{R}^d$ and its corresponding target, or label, t
 - ▶ **Regression**: t is a real number (e.g. stock price)
 - ▶ **Classification**: t is an element of a discrete set $\{1, \dots, C\}$

Denote the training set $\{(\mathbf{x}^{(1)}, t^{(1)}), \dots, (\mathbf{x}^{(N)}, t^{(N)})\}$

Nearest Neighbors

- Suppose we're given a novel input vector $\mathbf{x}$ we'd like to classify. The
- idea: find the nearest input vector to $\mathbf{x}$ in the training set $D_{tr} = \{(\mathbf{x}^{(1)}, t^{(1)}), \dots, (\mathbf{x}^{(N)}, t^{(N)})\}$ and copy its label.
- We can formalize “nearest” in terms of Euclidean distance (other choices possible).

$$\|\mathbf{x}^{(a)} - \mathbf{x}^{(b)}\|_2 = \sqrt{\sum_{j=1}^d (x_j^{(a)} - x_j^{(b)})^2}$$

Algorithm:

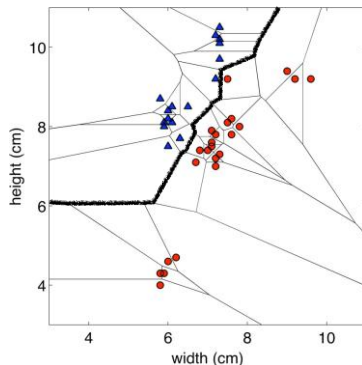
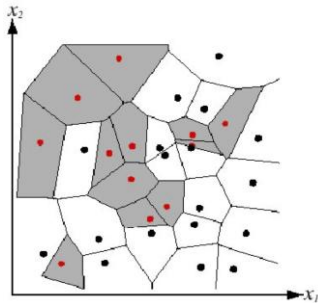
1. Find example $(\underline{\mathbf{x}^*}, t^*)$ (from the stored training set) closest to $\mathbf{x}$. That is:

$$\mathbf{x}^* = \underset{\mathbf{x}^{(i)} \in \text{training set}}{\operatorname{argmin}} \quad \text{distance}(\mathbf{x}^{(i)}, \mathbf{x})$$

2. Output $y = t^*$

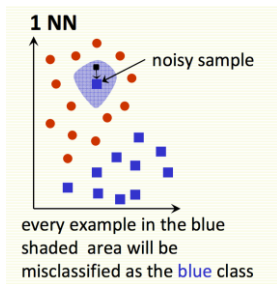
Nearest Neighbors: Decision Boundaries

- **Decision boundary:** a line or surface that divides different groups in a classification task.
- It shows which area belong to which class based on what the model decides.



Nearest Neighbors

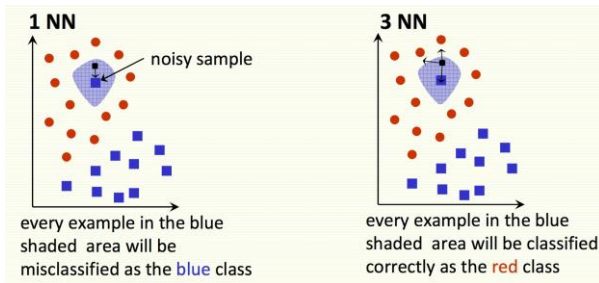
- Sensitive to noise or mis-labeled data (“class noise”). Solution?



[Pic by Olga Veksler]

Nearest Neighbors

- Sensitive to noise or mis-labeled data (“class noise”). Solution?
- Smooth by having k nearest neighbors vote



[Pic by Olga Veksler]

K-Nearest Neighbors

- Nearest neighbors **sensitive to noise or mis-labeled data** (“class noise”).
Solution?
- Smooth by having k nearest neighbors vote Let
- L be the set of all class labels.

Algorithm (kNN):

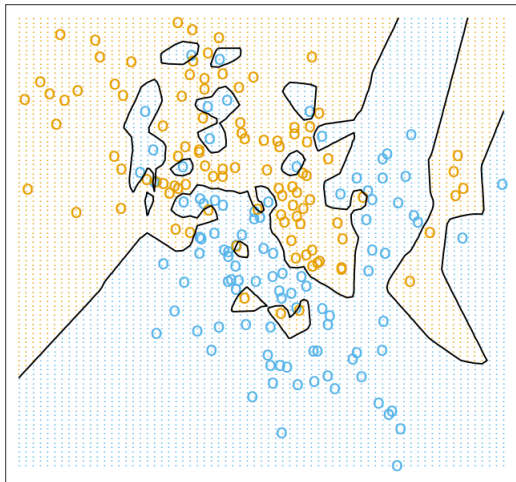
1. Find k examples $\{(\mathbf{x}^{(1)}, t^{(1)}), \dots, (\mathbf{x}^{(k)}, t^{(k)})\}$ in D_{tr} closest to the test instance $\mathbf{x}$
2. Classification output is majority class.

$$y^* = \max_{t^{(z)} \in \text{class labels}} \sum_{i=1}^k \mathbb{I}(t^{(z)} = t^{(i)})$$

$\mathbb{I}\{\text{statement}\}$ is the identity function and is equal to one whenever the statement is true. We could also write this as $\delta(t^{(z)}, t^{(i)})$, with $\delta(a, b) = 1$ if $a = b$, 0 otherwise.

K-Nearest Neighbors

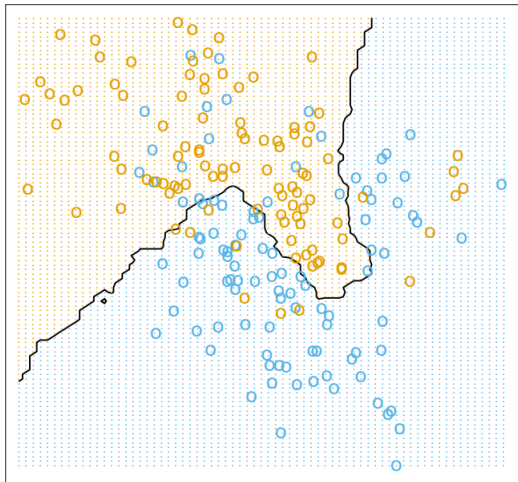
$k=1$



[Image credit: "The Elements of Statistical Learning"]

K-Nearest Neighbors

$k=15$

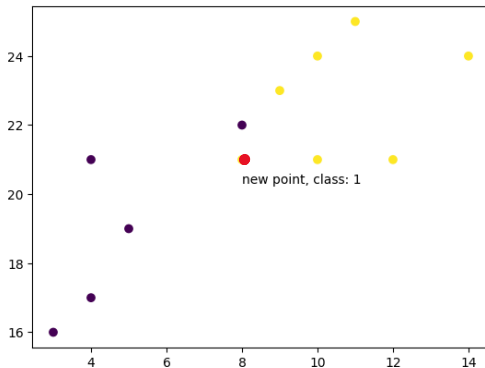


[Image credit: "The Elements of Statistical Learning"]

Example

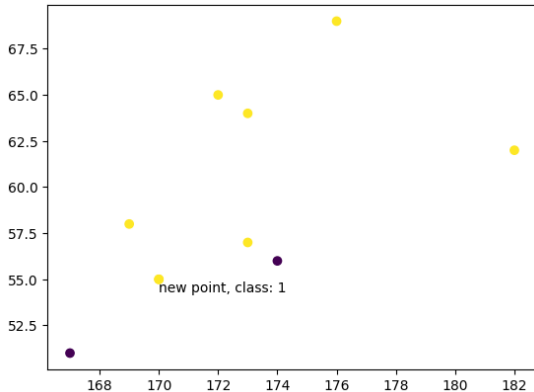
```
x = [4, 5, 10, 4, 3, 11, 14, 9, 8, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 23, 22, 21, 21]
classes = [0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1]
```

```
new_x = 8
new_y = 21
```



Example

Height (CM)	Weight (KG)	Class
167	51	Underweight
182	62	Normal
176	69	Normal
173	64	Normal
172	65	Normal
174	56	Underweight
169	58	Normal
173	57	Normal
170	55	Normal
170	57	?



Example

```
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier

x = [167, 182, 176, 173, 172, 174, 169, 173, 170]
y = [51, 62, 69, 64, 65, 56, 58, 57, 55]
classes = [0, 1, 1, 1, 1, 0, 1, 1, 1]
data = list(zip(x, y))
knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(data, classes)
new_x = 170
new_y = 55
new_point = [(new_x, new_y)]

prediction = knn.predict(new_point)
plt.scatter(x + [new_x], y + [new_y], c=classes +
[prediction[0]])
plt.text(x=new_x, y=new_y-0.7, s=f"new point, class:
{prediction[0]}")
plt.show()
```

Tradeoffs in choosing k ?

- Small k
 - ▶ Good at capturing fine-grained patterns
 - ▶ May **overfit**, i.e. be sensitive to random abnormalities in the training data
- Large k
 - ▶ Makes stable predictions by averaging over lots of examples
 - ▶ May **underfit**, i.e. fail to capture important regularities
- Balancing k
 - ▶ Optimal choice of k depends on number of data points n .

$\underline{k}$

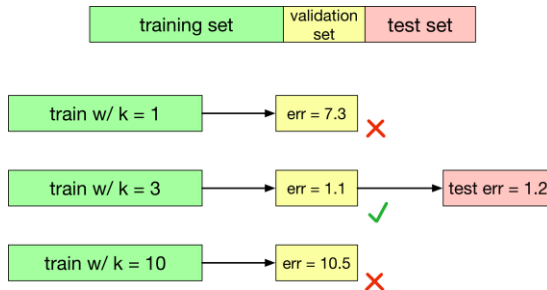
K-Nearest Neighbors

- We would like our algorithm to **generalize** to data it hasn't seen before.
- We can measure the **generalization error** (error rate on new examples) using a **test set**.

[Image credit: "The Elements of Statistical Learning"]

Validation and Test Sets

- k is an example of a **hyperparameter**, something we can't fit as part of the learning algorithm itself
- We can tune hyperparameters using a **validation set**:



- The test set is used only at the very end, to measure the generalization performance of the final configuration.

Pitfalls: The Curse of Dimensionality

- The Curse of Dimensionality refers to various phenomena that arise when dealing with high-dimensional data.
- As the number of features or dimensions increases, the volume of the feature space grows exponentially, leading to sparsity in the data distribution.
- This sparsity can result in several challenges such as increased computational complexity, overfitting, and deteriorating performance of certain algorithms.

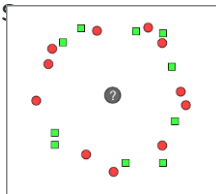
Pitfalls: The Curse of Dimensionality

The impact of dimensionality on the performance of KNN (K-Nearest Neighbors)

- **Increased Sparsity:** As the number of dimensions increases, the volume of the space grows exponentially. Consequently, the available data becomes sparser, meaning that data points are spread farther apart from each other.

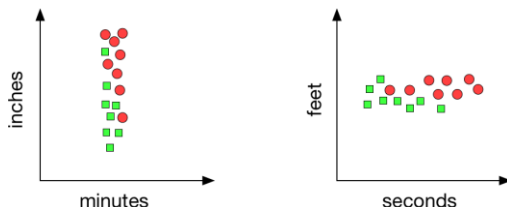
Pitfalls: The Curse of Dimensionality

- Nearest neighbors says that if two points are close in input space, their outputs must be close (the same).
- As dimension increases, all points appear equidistance – so how do we select the label for a test point?
- **Equal Distances:** In high-dimensional spaces, the concept of distance becomes less meaningful. As the number of dimensions increases, the distance between any two points tends to become more uniform, or equidistant.



Pitfalls: Normalization

- Nearest neighbors can be sensitive to the ranges of different features.
- Often, the units are arbitrary:



- Simple fix: **normalize** each dimension to be zero mean and unit variance. I.e., compute the mean μ_j and standard deviation σ_j , and take

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

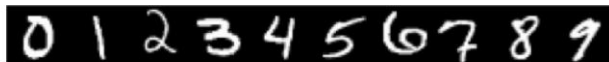
- Caution: depending on the problem, the scale might be important!

Pitfalls: Computational Cost

- Number of computations at **training time**: 0
- Number of computations at **test time**, per query (naïve algorithm)
 - ▶ Calculate D -dimensional Euclidean distances with N data points: $O(ND)$
 - ▶ Sort the distances: $O(N \log N)$
- This must be done for *each* query, which is very expensive by the standards of a learning algorithm!
- Need to store the entire dataset in memory!
- Tons of work has gone into algorithms and data structures for efficient nearest neighbors with high dimensions and/or large datasets.

Example: Digit Classification

- Decent performance when lots of data

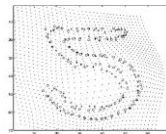
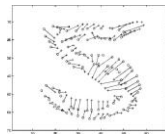
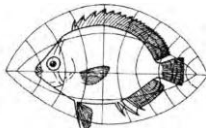
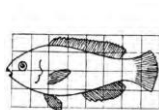


- Yann LeCunn – MNIST Digit Recognition
 - Handwritten digits
 - 28x28 pixel images: $d = 784$
 - 60,000 training samples
 - 10,000 test samples
- Nearest neighbour is competitive

	Test Error Rate (%)
Linear classifier (1-layer NN)	12.0
K-nearest-neighbors, Euclidean	5.0
K-nearest-neighbors, Euclidean, deskewed	2.4
K-NN, Tangent Distance, 16x16	1.1
K-NN, shape context matching	0.67
1000 RBF + linear classifier	3.6
SVM deg 4 polynomial	1.1
2-layer NN, 300 hidden units	4.7
2-layer NN, 300 HU, [deskewing]	1.6
LeNet-5, [distortions]	0.8
Boosted LeNet-4, [distortions]	0.7

Example: Digit Classification

- KNN can perform a lot better with a good similarity measure.
- Example: shape contexts for object recognition. In order to achieve invariance to image transformations, they tried to warp one image to match the other image.
 - ▶ Distance measure: average distance between corresponding points on *warped* images
- Achieved 0.63% error on MNIST, compared with 3% for Euclidean KNN.
- Competitive with conv nets at the time, but required careful engineering.



[Belongie, Malik, and Puzicha, 2002. Shape matching and object recognition using shape contexts.]

Example: 80 Million Tiny Images

- 80 Million Tiny Images was the first extremely large image dataset. It consisted of color images scaled down to 32×32 .

With a large dataset, you can

- find much better semantic matches.

Note: this required a carefully

- chosen similarity metric.



[Torralba, Fergus, and Freeman, 2007. 80 Million Tiny Images.]

Here are some prompts (tried out on GPT4) to revise some of the content you learned in lecture today.

- Explain the k-nearest neighbor algorithm to me as if I were a (ten year old) or (first year undergraduate student) or (without using any mathematics) or (using formal notation).
- What is the curse of dimensionality in machine learning?
- Create pseudocode for the k-nearest neighbor algorithm. (After model output.) Go line by line and explain what each line does to me.

Conclusions

- Simple algorithm that does all its work at test time — in a sense, no learning!
- Can control the complexity by varying k
- Suffers from the Curse of Dimensionality
- Next time: parametric models, which learn a compact summary of the data rather than referring back to it at test time.